

Lab 2: GDB

Student: Ilya-Linh Nguyen

Innopolis main: i.nguen@innopolis.university

Group: CBS-01

Task

- Learn these utilities: GDB, strace, GHIDRA, ldd.
- **Hack** the application provided by the instructor!
- Application requires the license that is generated based on your hardware id.
- License will be stored somewhere on your PC (filesystem).
- Implement 2 artifacts:
 1. Create keygen for this app.
 2. Create binary patch to disable licensing completely.

Solution

Here you can see the files of my solution:

- GitHub URL: <https://github.com/ilyalinhnguyen/adv-linux>

For this lab I switched from my macOS to the Linux laptop for more comfortable lab completion.

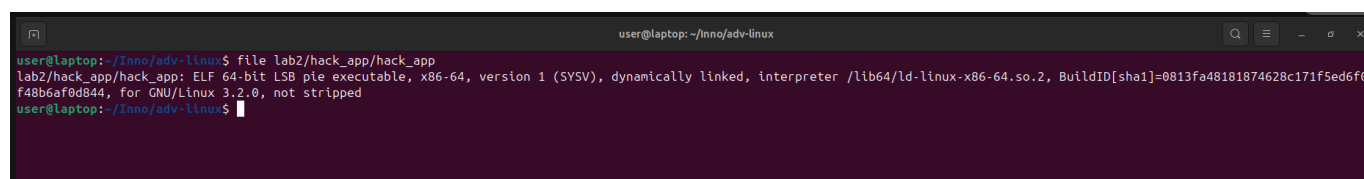
Initial Triage

File exploring and dependency check

Firstly, let's inspect our app with some common commands:

```
file lab2/hack_app/hack_app
```

Result:

A terminal window with a dark background and light green text. The title bar reads 'user@laptop: ~/Inno/adv-linux'. The command 'file lab2/hack_app/hack_app' has been executed. The output is: 'lab2/hack_app/hack_app: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=0813fa48181874628c171f5ed6f0f49b6af0d844, for GNU/Linux 3.2.0, not stripped'.

```
user@laptop:~/Inno/adv-linux$ file lab2/hack_app/hack_app
lab2/hack_app/hack_app: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=0813fa48181874628c171f5ed6f0f49b6af0d844, for GNU/Linux 3.2.0, not stripped
user@laptop:~/Inno/adv-linux$
```

```
ldd lab2/hack_app/hack_app
```

```
user@laptop:~/Inno/adv-linux$ ldd lab2/hack_app/hack_app
linux-vdso.so.1 (0x0000762c9999a000)
libcrypto.so.1.1 => not found
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x0000762c99600000)
/lib64/ld-linux-x86-64.so.2 (0x0000762c9999c000)
user@laptop:~/Inno/adv-linux$
```

Key findings:

- Binary is 64-bit ELF PIE, dynamically linked.
- Missing dependency:
 - `libcrypto.so.1.1 => not found`

Where this comes from: the binary was compiled against OpenSSL 1.1 and imports MD5 symbols from `libcrypto.so.1.1`. I verified the required imported symbols from the dynamic symbol table:

```
objdump -T lab2/hack_app/hack_app | grep -E "MD5_(Init|Update|Final)"
```

```
user@laptop:~/Inno/adv-linux$ objdump -T lab2/hack_app/hack_app | grep -E "MD5_(Init|Update|Final)"
0000000000000000      DF *UND* 0000000000000000 (OPENSSL_1_1_0) MD5_Final
0000000000000000      DF *UND* 0000000000000000 (OPENSSL_1_1_0) MD5_Update
0000000000000000      DF *UND* 0000000000000000 (OPENSSL_1_1_0) MD5_Init
user@laptop:~/Inno/adv-linux$
```

Let's fix this now, to prevent errors in future:

I created a local compatibility shared library that exports only the needed MD5 functions with the correct `OPENSSL_1_1_0` symbol version (so the loader can resolve them):

- Source: `lab2/deps/md5_compat.c`
- Version script: `lab2/deps/openssl_1_1.map`
- Output: `lab2/deps/libcrypto.so.1.1`

Build command (chosen by me to compile the shim and apply symbol versioning via the version script):

```
gcc -shared -fPIC lab2/deps/md5_compat.c \
  -Wl,--version-script=lab2/deps/openssl_1_1.map \
  -Wl,-soname,libcrypto.so.1.1 \
  -o lab2/deps/libcrypto.so.1.1
```

Then I will execute the target app with:

```
LD_LIBRARY_PATH=/home/user/Inno/adv-linux/lab2/deps ...
```

Let's check with `string` command now:

```
strings lab2/hack_app/hack_app
```

```
user@laptop:~/Inno/adv-linux$ strings lab2/hack_app/hack_app
/lib64/ld-linux-x86-64.so.2
libcrypto.so.1.1
__gmon_start__
_ITM_deregisterTMCloneTable
_ITM_registerTMCloneTable
MD5_Final
MD5_Init
MD5_Update
libc.so.6
sprintf
strncmp
__isoc99_scanf
puts
__stack_chk_fail
setxattr
system
readlink
__cxa_finalize
getxattr
__libc_start_main
snprintf
OPENSSL_1_1_0
GLIBC_2.7
GLIBC_2.4
GLIBC_2.3
GLIBC_2.2.5
u+UH
[]A\A]A^A_
%08X%08X
%02x
/proc/self/exe
user.license
Welcome to Lab2 super secure program!
Your HWID is %08X%08X.
Enter the license key:
%32s
Now you app is activated! Thanks for purchasing!
Provided key is wrong! App is closing!
Your app is licensed to this PC!
read -p 'Press Enter to continue...' var
:*3$"
GCC: (Ubuntu 9.4.0-1ubuntu1~20.04.1) 9.4.0
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.8061
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
```

Super interesting strings found:

- /proc/self/exe
- user.license
- Your HWID is %08X%08X.
- Enter the license key:
- Your app is licensed to this PC!
- Now you app is activated! Thanks for purchasing!
- Provided key is wrong! App is closing!
- MD5_Init, MD5_Update, MD5_Final

These strings already hint that:

- App computes a hardware-based ID (**HWID**).
- Key is MD5-related.
- License is stored in extended attribute (**user.license**) on the executable file.

Static Reverse Engineering (**gdb** disassembly)

I disassembled **main** in batch mode:

```
gdb -q -batch -ex "file lab2/hack_app/hack_app" -ex "disassemble main"
```

The most valuable info that I could find from **main**

Calls **__get_cpuid(1, &eax, &ebx, &ecx, &edx)**.

```
0x00000000000001456 <+114>: call    0x131e <__get_cpuid>
0x0000000000000145b <+119>: mov     -0x2c(%rbp),%eax
0x0000000000000145e <+122>: shr     $0x18,%eax
0x00000000000001461 <+125>: mov     %eax,%edx
0x00000000000001463 <+127>: mov     -0x2c(%rbp),%eax
0x00000000000001466 <+130>: shl     $0x8,%eax
```

Performs byte swap on **eax** and **edx**.

From screenshot earlier:

```
0x0000000000000145b <+119>: mov     -0x2c(%rbp),%eax
0x0000000000000145e <+122>: shr     $0x18,%eax
```

And

```
0x00000000000001485 <+161>: mov     %eax,-0x10(%rbp)
...
0x000000000000014b2 <+206>: mov     %eax,-0xc(%rbp)
```

from

```

0x0000000000001485 <+161>: mov    %eax,-0x10(%rbp)
0x0000000000001488 <+164>: mov    -0x20(%rbp),%eax
0x000000000000148b <+167>: shr    $0x18,%eax
0x000000000000148e <+170>: mov    %eax,%edx
0x0000000000001490 <+172>: mov    -0x20(%rbp),%eax
0x0000000000001493 <+175>: shl    $0x8,%eax
0x0000000000001496 <+178>: and    $0xff0000,%eax
0x000000000000149b <+183>: or     %eax,%edx
0x000000000000149d <+185>: mov    -0x20(%rbp),%eax
0x00000000000014a0 <+188>: shr    $0x8,%eax
0x00000000000014a3 <+191>: and    $0xff00,%eax
0x00000000000014a8 <+196>: or     %eax,%edx
0x00000000000014aa <+198>: mov    -0x20(%rbp),%eax
0x00000000000014ad <+201>: shl    $0x18,%eax
0x00000000000014b0 <+204>: or     %edx,%eax
0x00000000000014b2 <+206>: mov    %eax,-0xc(%rbp)

```

Builds HWID string with:

- `snprintf(PSN, 0x11, "%08X%08X", swapped_eax, swapped_edx).`

```

0x00000000000014c0 <+220>: lea    0xb41(%rip),%rdx    # 0x2008
0x00000000000014c7 <+227>: mov    $0x11,%esi
0x00000000000014cc <+232>: lea    0x4bdd(%rip),%rdi    # 0x60b0 <PSN>
0x00000000000014d3 <+239>: mov    $0x0,%eax
0x00000000000014d8 <+244>: call   0x1130 <snprintf@plt>

```

Computes MD5 of this 16-char ASCII string.

```

0x00000000000014e9 <+261>: call   0x1397 <calc_md5>
0x00000000000014ee <+266>: movl   $0x0,-0x18(%rbp)
0x00000000000014f5 <+273>: jmp     0x153b <main+343>
0x00000000000014f7 <+275>: mov     $0xf,%eax
0x00000000000014fc <+280>: sub     -0x18(%rbp),%eax

```

Builds expected key as lowercase hex of **reversed MD5 digest bytes**.

The index `0xf - i` used to read bytes from `md5digest`:

```

0x00000000000014f7 <+275>: mov     $0xf,%eax
0x00000000000014fc <+280>: sub     -0x18(%rbp),%eax
0x00000000000014ff <+283>: cltq
0x0000000000001501 <+285>: lea     0x3b68(%rip),%rdx    # 0x5070 <md5digest>
0x0000000000001508 <+292>: movzbl (%rax,%rdx,1),%eax
0x000000000000150c <+296>: movzbl %al,%eax
0x000000000000150f <+299>: mov     -0x18(%rbp),%edx
0x0000000000001512 <+302>: add     %edx,%edx
0x0000000000001514 <+304>: movslq %edx,%rdx
0x0000000000001517 <+307>: lea     0x3b22(%rip),%rcx    # 0x5040 <md5decode>
0x000000000000151e <+314>: add     %rdx,%rcx
0x0000000000001521 <+317>: mov     %eax,%edx
0x0000000000001523 <+319>: lea     0xae7(%rip),%rsi     # 0x2011
0x000000000000152a <+326>: mov     %rcx,%rdi
0x000000000000152d <+329>: mov     $0x0,%eax
0x0000000000001532 <+334>: call    0x1190 <sprintf@plt>

```

Then from:

```

0x0000000000001554 <+368>: call    0x11b0 <readlink@plt>
0x0000000000001559 <+373>: mov     $0x1000,%ecx
0x000000000000155e <+378>: lea     0x2adb(%rip),%rdx      # 0x4040 <xattrValue>
0x0000000000001565 <+385>: lea     0xab9(%rip),%rsi       # 0x2025
0x000000000000156c <+392>: lea     0x3b0d(%rip),%rdi      # 0x5080 <binaryPath>
0x0000000000001573 <+399>: call    0x11d0 <getxattr@plt>
0x0000000000001578 <+404>: lea     0xab9(%rip),%rdi       # 0x2038
0x000000000000157f <+411>: call    0x1140 <puts@plt>
0x0000000000001584 <+416>: mov     $0x21,%edx
0x0000000000001589 <+421>: lea     0x2ab0(%rip),%rsi      # 0x4040 <xattrValue>
0x0000000000001590 <+428>: lea     0x3aa9(%rip),%rdi      # 0x5040 <md5decode>
0x0000000000001597 <+435>: call    0x1150 <strncmp@plt>
0x000000000000159c <+440>: test    %eax,%eax
0x000000000000159e <+442>: jne     0x15a7 <main+451>
0x00000000000015a0 <+444>: movl    $0x1,-0x1c(%rbp)
0x00000000000015a7 <+451>: cmpl    $0x0,-0x1c(%rbp)
0x00000000000015ab <+455>: jne     0x163f <main+603>
0x00000000000015b1 <+461>: mov     -0xc(%rbp),%edx
0x00000000000015b4 <+464>: mov     -0x10(%rbp),%eax
0x00000000000015b7 <+467>: mov     %eax,%esi
0x00000000000015b9 <+469>: lea     0xaa0(%rip),%rdi       # 0x2060
0x00000000000015c0 <+476>: mov     $0x0,%eax
0x00000000000015c5 <+481>: call    0x1120 <printf@plt>
0x00000000000015ca <+486>: lea     0x4aaf(%rip),%rsi      # 0x6080 <userInput>
0x00000000000015d1 <+493>: lea     0xab7(%rip),%rdi       # 0x208f
0x00000000000015d8 <+500>: mov     $0x0,%eax
0x00000000000015dd <+505>: call    0x11f0 <__isoc99_scanf@plt>
0x00000000000015e2 <+510>: mov     $0x21,%edx
0x00000000000015e7 <+515>: lea     0x4a92(%rip),%rsi      # 0x6080 <userInput>
0x00000000000015ee <+522>: lea     0x3a4b(%rip),%rdi      # 0x5040 <md5decode>
0x00000000000015f5 <+529>: call    0x1150 <strncmp@plt>
0x00000000000015fa <+534>: test    %eax,%eax
0x00000000000015fc <+536>: jne     0x1631 <main+589>
0x00000000000015fe <+538>: mov     $0x0,%r8d
0x0000000000001604 <+544>: mov     $0x21,%ecx
0x0000000000001609 <+549>: lea     0x3a30(%rip),%rdx      # 0x5040 <md5decode>
0x0000000000001610 <+556>: lea     0xa0e(%rip),%rsi       # 0x2025
0x0000000000001617 <+563>: lea     0x3a62(%rip),%rdi      # 0x5080 <binaryPath>
0x000000000000161e <+570>: call    0x11a0 <setxattr@plt>

```

We got:

Reads current executable path via:

- `readlink("/proc/self/exe", ...).`

```
0x0000000000001554 <+368>: call    0x11b0 <readlink@plt>
```

Reads xattr:

- `getxattr(binaryPath, "user.license", ...).`

```
0x0000000000001573 <+399>: call    0x11d0 <getxattr@plt>
```

Compares expected key and xattr with:

- `strncmp(expected, xattrValue, 0x21).`

```
0x00000000000001597 <+435>: call    0x1150 <strncmp@plt>
```

If not licensed, asks user input (`scanf("%32s")`), compares, and if correct:

- `setxattr(binaryPath, "user.license", expected, 0x21, 0).`

```
0x000000000000015dd <+505>: call    0x11f0 <__isoc99_scanf@plt>
...
0x0000000000000161e <+570>: call    0x11a0 <setxattr@plt>
```

License gate branch in `main`:

- Original instruction at virtual/file offset `0x15AB`:
 - `0f 85 8e 00 00 00` (`jne 0x163f`)

```
0x000000000000015ab <+455>: jne     0x163f <main+603>
```

Patch strategy:

- Replace conditional jump with unconditional jump to licensed branch:
 - New bytes: `e9 8f 00 00 00 90` (`jmp 0x163f; nop`)

Keygen Implementation

Implementing the python `lab2/keygen.py` script for making key. The app constructs the expected license key fully inside `main`:

- It queries CPU identification data via CPUID leaf 1:
 - `call __get_cpuid` with function id `1`
- It then uses only **two 32-bit values** from that leaf:
 - `eax` and `edx`
- Both values are converted to big-endian (byte-swapped).
- The program formats HWID as a 16-character uppercase hex string:
 - `HWID = "%08X%08X" % (bswap32(eax), bswap32(edx))`
- It computes MD5 of that **ASCII HWID string**.
- Finally, it creates the license string by taking the **MD5 digest bytes in reverse order** and printing each byte as lowercase hex:
 - `key = hex(md5(HWID))[::-1 by bytes]`

That matches the disassembly results:

```
0x00000000000001456 <+114>: call    0x131e <__get_cpuid>
...
```

```

0x000000000000014d8 <+244>: call    0x1130 <snprintf@plt>      ; "%08X%08X"
0x000000000000014e9 <+261>: call    0x1397 <calc_md5>
...
0x000000000000014f7 <+275>: mov     $0xf,%eax
0x000000000000014fc <+280>: sub     -0x18(%rbp),%eax      ; index = 0xF -
i
0x00000000000001508 <+292>: movzbl  (%rax,%rdx,1),%eax    ; md5digest[15-
i]
0x00000000000001532 <+334>: call    0x1190 <sprintf@plt>   ; "%02x"

```

How `keygen.py` reproduces it

Step-by-step mapping to the recovered logic:

1. **Read CPUID leaf 1** In `keygen.py`, I execute the `cputid` instruction (leaf = 1) via a small in-memory code stub using `mmap` (RWX page) and `ctypes`. This returns the same 4 registers the binary uses: `eax`, `ebx`, `ecx`, `edx`.
2. **Compute the HWID string exactly like the binary** Only `eax` and `edx` are used.
 - `hwid_hi = bswap32(eax)`
 - `hwid_lo = bswap32(edx)`
 - `HWID = f"{hwid_hi:08X}{hwid_lo:08X}"` This matches the binary's `snprintf("%08X%08X", ...)`.
3. **Compute MD5 over the ASCII HWID** `digest = hashlib.md5(HWID.encode("ascii")).digest()`
4. **Reverse digest bytes and hex-encode** The binary reads bytes in the order `md5digest[15]`, `md5digest[14]`, ..., `md5digest[0]`.
So the keygen does:
 - `key = "".join(f"{b:02x}" for b in digest[::-1])`

This produces a **32-character** lowercase hex key, which is exactly what the program reads with `scanf("%32s")`.

Run:

```
python3 lab2/keygen.py
```

Observed output on my Linux laptop:

```

user@laptop:~/Inno/adv-linux$ python3 lab2/keygen.py
HWID: 000FA500FFFB8B17
License key: 37e9ef58cd7826a3379678653268ed54
user@laptop:~/Inno/adv-linux$

```

Binary Patch Implementation

The app has two high-level states:

- **Licensed path:** prints `Your app is licensed to this PC!` and exits (no prompt).
- **Unlicensed path:** prints HWID and asks for a key; if correct, it writes `user.license` xattr.

The "gate" that selects which path to execute is a conditional jump:

```
0x000000000000015ab <+455>: jne    0x163f <main+603>
```

The report's earlier finding shows that `-0x1c(%rbp)` is set to 1 when the xattr matches the expected key, then the program branches to the licensed message.

At `0x15AB`, the original instruction bytes are:

- `0f 85 8e 00 00 00 = jne rel32`

So if the condition is true (licensed), execution jumps to `0x163f` (licensed branch).

To disable licensing completely, we force execution to always go to that branch, regardless of the condition:

- Replace `jne` with `jmp rel32` to the same target.

Patched bytes:

- `e9 8f 00 00 00 90`
 - `e9 8f 00 00 00 = jmp rel32` (to `0x163f`)
 - `90 = nop` (fills remaining 1 byte because `jmp rel32` is 5 bytes, while `jne rel32` is 6 bytes)

So the patched control flow always behaves as "licensed".

What `patch.py` script does

Behavior:

- Checks bytes at offset `0x15AB`.
- Verifies expected original bytes (`0f858e000000`) to avoid accidental corruption.
- Replaces with patched bytes (`e98f00000090`).
- Supports idempotence (if already patched, prints message and exits).

Usage:

```
cp lab2/hack_app/hack_app lab2/hack_app/hack_app.patched
python3 lab2/patch.py lab2/hack_app/hack_app.patched
```

And let's disassembly this and check:

```
objdump -d -M intel lab2/hack_app/hack_app.patched | sed -n '418,434p'
```

```
user@laptop:~/Inno/adv-linux$ objdump -d -M intel lab2/hack_app/hack_app.patched | sed -n '418,434p'
1584:    ba 21 00 00 00    mov     edx,0x21
1589:    48 8d 35 b0 2a 00 00 lea     rsi,[rip+0x2ab0]      # 4040 <xattrValue>
1590:    48 8d 3d a9 3a 00 00 lea     rdi,[rip+0x3aa9]      # 5040 <md5decode>
1597:    e8 b4 fb ff ff    call    1150 <strncmp@plt>
159c:    85 c0             test    eax,eax
159e:    75 07             jne     15a7 <main+0x1c3>
15a0:    c7 45 e4 01 00 00 00 mov     DWORD PTR [rbp-0x1c],0x1
15a7:    83 7d e4 00       cmp     DWORD PTR [rbp-0x1c],0x0
15ab:    e9 8f 00 00 00    jmp     163f <main+0x25b>
15b0:    90               nop
15b1:    8b 55 f4         mov     edx,DWORD PTR [rbp-0xc]
15b4:    8b 45 f0         mov     eax,DWORD PTR [rbp-0x10]
15b7:    89 c6             mov     esi,eax
15b9:    48 8d 3d a0 0a 00 00 lea     rdi,[rip+0xaa0]      # 2060 <_IO_stdin_used+0x60>
15c0:    b8 00 00 00 00    mov     eax,0x0
15c5:    e8 56 fb ff ff    call    1120 <printf@plt>
15ca:    48 8d 35 af 4a 00 00 lea     rsi,[rip+0x4aaf]      # 6080 <userInput>
user@laptop:~/Inno/adv-linux$
```

At 0x15ab: jmp 0x163f

This forces control flow into licensed branch and bypasses normal license check path.

A little of testing

Let's try wrong key for the app

```
cp lab2/hack_app/hack_app lab2/hack_app/hack_app.clean
printf "advlinuxadvlinuxadvlinuxadvlinux\n\n" | \
LD_LIBRARY_PATH=/home/user/Inno/adv-linux/lab2/deps \
lab2/hack_app/hack_app.clean
```

Output:

```
user@laptop:~/Inno/adv-linux$ cp lab2/hack_app/hack_app lab2/hack_app/hack_app.clean
printf "advlinuxadvlinuxadvlinuxadvlinux\n\n" | \
LD_LIBRARY_PATH=/home/user/Inno/adv-linux/lab2/deps \
lab2/hack_app/hack_app.clean
Welcome to Lab2 super secure program!
Your HWID is 000FA500FFFB8B17.
Enter the license key: Provided key is wrong! App is closing!
user@laptop:~/Inno/adv-linux$
```

And now put correct key

```
printf "37e9ef58cd7826a3379678653268ed54\n\n" | \
LD_LIBRARY_PATH=/home/user/Inno/adv-linux/lab2/deps \
lab2/hack_app/hack_app.clean
```

Output:

```
user@laptop:~/Inno/adv-linux$ printf "37e9ef58cd7826a3379678653268ed54\n\n" | \
LD_LIBRARY_PATH=/home/user/Inno/adv-linux/lab2/deps \
lab2/hack_app/hack_app.clean
Welcome to Lab2 super secure program!
Your HWID is 000FA500FFFB8B17.
Enter the license key: Now you app is activated! Thanks for purchasing!
user@laptop:~/Inno/adv-linux$
```

Now let's just run after activation

```
printf "\n" | LD_LIBRARY_PATH=/home/user/Inno/adv-linux/lab2/deps \
lab2/hack_app/hack_app.clean
```

Output:

```
user@laptop:~/Inno/adv-linux$ printf "\n" | LD_LIBRARY_PATH=/home/user/Inno/adv-linux/lab2/deps \
lab2/hack_app/hack_app.clean
Welcome to Lab2 super secure program!
Your app is licensed to this PC!
user@laptop:~/Inno/adv-linux$
```

That's it for this task